

1. Differentiate Checked and Unchecked Exceptions (Interview Notes)

What are Checked Exceptions?

Checked exceptions are **checked by the compiler**.

The compiler forces you to either:

- Handle them using try-catch, or
- Declare them using throws.

Example:

```
FileReader file = new FileReader("data.txt");
```

This causes a compile-time error unless handled.

Examples:

- IOException
 - SQLException
 - FileNotFoundException
-

What are Unchecked Exceptions?

Unchecked exceptions occur **at runtime**.

The compiler does **not** force you to handle them.

They usually occur because of programming mistakes.

Examples:

- NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException
-

Checked vs Unchecked

Checked Exception	Unchecked Exception
Checked by compiler	Occur at runtime
Must be handled or declared	Handling is optional
Extend Exception	Extend RuntimeException
Usually external problems	Usually programming mistakes

Important Interview Points

- Checked exceptions are verified at **compile time**.
- Unchecked exceptions occur at **runtime**.
- Checked exceptions must be handled using **try-catch** or **throws**.
- Unchecked exceptions are subclasses of **RuntimeException**.
- Checked exceptions usually represent **recoverable external conditions**, while unchecked exceptions usually indicate **bugs in the code**.
- Good programs aim to **prevent** runtime exceptions rather than simply catching them.

Tricky Interview Questions

Q1. Which exceptions must be handled?

Answer:

Checked exceptions.

Q2. Is RuntimeException checked?

Answer: No.

It is an unchecked exception.

Q3. Is IOException checked or unchecked?

Answer:

Checked.

Q4. Why are unchecked exceptions not checked by the compiler?

Answer:

Because they usually result from programming errors that should be fixed rather than forced to be caught.

Q5. Can unchecked exceptions be caught?

Answer: Yes.

Using try-catch, although preventing them is usually better.

Interview Summary

- **Checked exceptions are compile-time checked and must be handled or declared.**
- **Unchecked exceptions occur at runtime and are not required to be handled.**
- Checked exceptions represent **recoverable external problems**.
- Unchecked exceptions usually indicate **programming mistakes**.
- Knowing the difference helps decide **when to handle an exception and when to fix the code instead**.

Interview One-Liner:

"Checked exceptions must be handled or declared because the compiler enforces them, while unchecked exceptions are runtime errors caused mainly by programming mistakes."

2. Use try-catch for Recoverable Errors (Interview Notes)

What is try-catch?

try-catch is used to **handle exceptions** so the program can continue running instead of terminating abruptly.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

Output

```
Cannot divide by zero
```

Why Use try-catch?

Use try-catch when an exception is **expected and recoverable**, such as:

- Reading a file
- Database operations
- User input
- Network communication

Execution Flow

```
try  
↓  
Exception?  
↓  
Yes → catch  
↓  
Continue Program
```

If no exception occurs:

```
try
```

```
↓
```

```
No Exception
```

```
↓
```

```
catch skipped
```

Important Interview Points

- Code that may throw an exception is placed inside the **try** block.
- The **catch** block executes only if a matching exception occurs.
- After handling the exception, the program continues with the next statement.
- Multiple catch blocks can handle different exception types.
- Catch the **most specific exception first**; otherwise, the compiler reports unreachable code.
- Use try-catch for **recoverable situations**, not as a replacement for proper validation.

Tricky Interview Questions

Q1. Can a try block exist without catch?

Answer:

Yes, if it has a finally block.

Q2. Can a catch block exist without try?

Answer: No.

Q3. Can we have multiple catch blocks?

Answer: Yes.

Q4. What happens if no exception occurs?

Answer:

The catch block is skipped.

Q5. Why should specific exceptions be caught before general ones?

Answer:

Because Java matches catch blocks from top to bottom. A general exception first would make specific catches unreachable.

Interview Summary

- try-catch allows a program to **handle recoverable exceptions** without crashing.
- The try block contains risky code, while the catch block handles matching exceptions.
- Multiple catch blocks are supported, but **specific exceptions must come before general ones**.
- Use exception handling for **exceptional situations**, not normal program flow.

Interview One-Liner:

"try-catch lets a program recover from expected exceptions by handling them gracefully instead of terminating execution."

3. Use finally for Cleanup Logic (Conceptually)

What is finally?

The finally block contains code that **always executes**, whether an exception occurs or not.

It is mainly used for **cleanup operations**.

```
try {  
    System.out.println("Reading file");  
} catch (Exception e) {
```

```
        System.out.println("Error");
    } finally {
        System.out.println("Closing file");
    }
```

Output (if exception occurs)

```
Reading file
Error
Closing file
```

Output (if no exception occurs)

```
Reading file
Closing file
```

Why Use finally?

Use finally to release resources such as:

- Closing files
- Closing database connections
- Closing sockets
- Releasing locks

This ensures resources are cleaned up even if an exception occurs.

Execution Flow

```
try
↓
Exception?
↓
```

```
catch (if needed)
```

↓

```
finally (Always Executes)
```

Important Interview Points

- The finally block executes **whether an exception occurs or not**.
 - It is primarily used for **resource cleanup**.
 - finally executes even if a return statement is encountered in try or catch (except in rare cases like JVM termination using System.exit()).
 - A try block can have a finally block without a catch block.
 - Modern Java often uses **try-with-resources** instead of finally for automatically closing resources.
-

Tricky Interview Questions

Q1. Does finally always execute?

Answer:

Almost always.

It may not execute if the JVM terminates abruptly (e.g., System.exit()).

Q2. Can we use finally without catch?

Answer: Yes.

```
try {  
    // code  
} finally {  
    // cleanup  
}
```

Q3. What is the main purpose of finally?

Answer:

To perform cleanup operations regardless of whether an exception occurs.

Q4. Does finally execute after a return statement?

Answer: Yes.

finally runs before the method actually returns.

Q5. What is preferred in modern Java for closing resources?

Answer:

Try-with-resources, because it closes resources automatically.

Interview Summary

- **finally is used for cleanup code that should run regardless of success or failure.**
- It commonly releases resources such as files, database connections, and network sockets.
- finally executes after try and catch, even when a return statement is present.
- In modern Java, **try-with-resources** is preferred for managing resources that implement AutoCloseable.

Interview One-Liner:

"finally ensures cleanup code executes regardless of whether an exception occurs, making it ideal for releasing resources such as files and database connections."